

Kubernetes Giriş Kursu

Detaylı Eğitimci Dokümantasyonu — 2 Gün

Süre	2 gün — toplam ≈14 saat (kurulum hariç)
Hedef kitle	Docker kullanan yazılım geliştiriciler
Lab ortamı	Önceden kurulu Minikube + kubectl
Format	Eğitmen anlatım + her bölümde uygulamalı lab
Hedef çıktı	Temel ve orta seviye K8s nesnelerini ezbere değil anlayarak kullanabilme

Doküman Yapısı

Her konu beş başlık altında işlenir:

- Amaç ve mantık — neden var, hangi sorunu çözüyor?
- Mimarideki yeri — diğer K8s nesneleri ile ilişkisi (diyagramla)
- Komutlar ve flag'ler — imperative kullanım (tablolar)
- Declarative — YAML manifest — üretimde tercih edilen yöntem
- İleri detaylar, best practice ve sık hatalar — eğitimci notları

Zaman Planı (Kurulum Hariç)

Kurulum (Minikube, kubectI, Docker doğrulaması) kurs öncesinde tamamlandığı varsayılır. İlk dakikadan itibaren mimari ile başlanır ve uygulamalı çalışma için en fazla zaman Pod, Deployment ve Service konularına ayrılır.

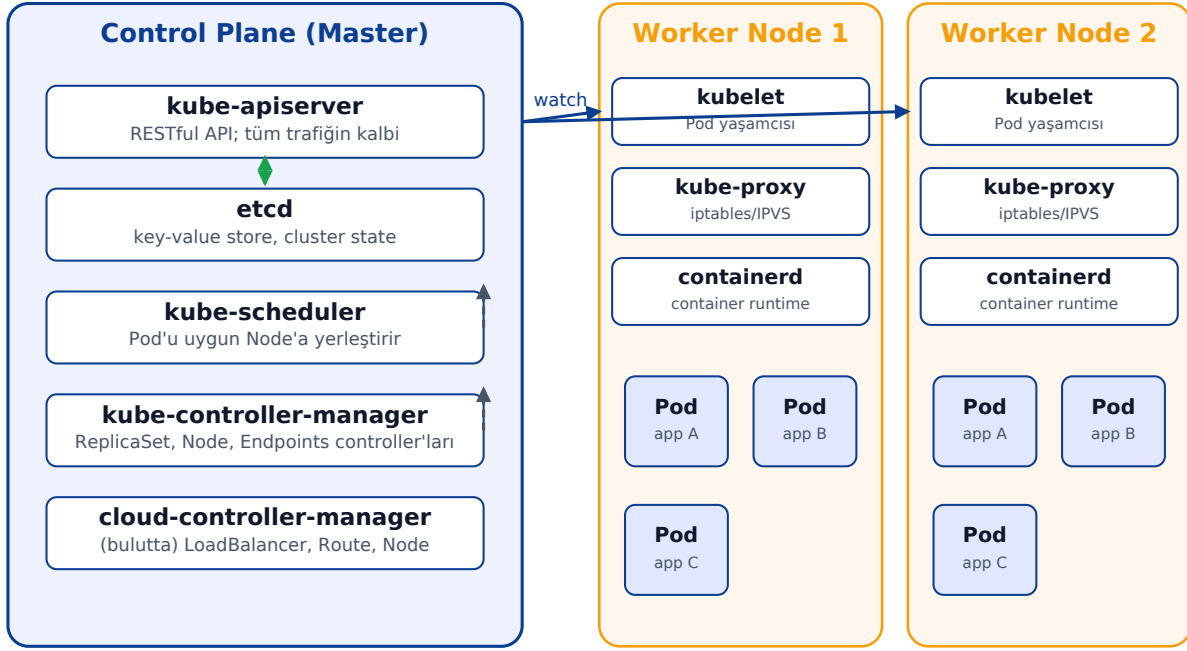
Modül	Süre	Açıklama
M1. Kubernetes mimarisi	45 dk	Anlatım + 2 mini lab
M2. Pod	120 dk	5 lab senaryosu
M3. ReplicaSet & Deployment	120 dk	Rolling update, rollback
M4. Service ve Ağ Modeli	120 dk	ClusterIP/NodePort/LoadBalancer
Toplam 1. gün	≈ 6 saat 45 dk	+ aralar
M5. ConfigMap & Secret	90 dk	envFrom, volume mount
M6. Volume, PV ve PVC	90 dk	emptyDir → PVC karşılaştırması
M7. Namespace, Label, Annotation	45 dk	RBAC scope'u
M8. Resource Yönetimi ve Probe'lar	75 dk	requests/limits + 3 probe
M9. Ingress	60 dk	Path/host routing
M10. Kapanış projesi & sorun giderme	90 dk	Tüm konuları birleştir
Toplam 2. gün	≈ 7 saat 30 dk	+ aralar

M1. Kubernetes Mimarisi (45 dk)

Kubernetes, deklaratif olarak belirtilen "istenen durumu" (desired state) sürekli olarak mevcut durumla karşılaştırıp farkı kapatmaya çalışan bir kontrol döngüsü sistemidir. Tüm bileşenler bu fikrin etrafında tasarlanmıştır.

Üst Seviye Görünüm

Kubernetes Mimarisi — Genel Görünüm



Tüm bileşenler etcd'ye yalnızca kube-apiserver üzerinden erişir.

Control Plane Bileşenleri — Detaylı

kube-apiserver

Cluster'ın tek giriş kapısıdır. kubectl, kubelet, scheduler, controller'lar, dashboard — herkes RESTful HTTP üzerinden yalnızca API server ile konuşur. API server'ın görevleri:

- REST endpoint'leri yayınlamak (örn. /api/v1/namespaces/default/pods).
- Kimlik doğrulama (authentication): mTLS sertifika, bearer token, OIDC.
- Yetkilendirme (authorization): RBAC, ABAC, Webhook.
- Admission control: kuralları (NamespaceLifecycle, ResourceQuota, MutatingWebhook, ValidatingWebhook) uygular; nesneyi reddedebilir veya değiştirebilir.
- etcd'ye yazma/okuma; etcd'ye doğrudan erişebilen tek bileşendir.
- Watch akışları: client'lar bir kaynağı izlerken API server değişiklikleri uzun süreli HTTP bağlantısı üzerinden push eder.

etcd

Raft konsensüs algoritması kullanan, tutarlı, dağıtık bir key-value store'dur. Cluster'ın tüm durumu (Pod, Service, ConfigMap, Secret, RBAC, vs.) burada saklanır. Birkaç önemli detay:

- Üretim kurulumlarında genellikle 3 veya 5 etcd üyesi (odd sayı) kullanılır; çoğunluk kaybı = cluster down.
- Yedek alınmazsa cluster geri getirilemez (etcdctl snapshot save).
- Encryption at rest default kapalıdır; üretimde mutlaka açın (KMS provider).
- Yalnızca kube-apiserver'a güvenir; doğrudan etcd'ye erişim üretimde tehlikelidir.

kube-scheduler

Henüz spec.nodeName atanmamış Pod'ları izler ve onlara uygun Node'u seçer. İki ana evreden oluşur:

- Filtering (predicate): Hangi Node'lar bu Pod'u çalıştırabilir? Yeterli CPU/RAM, uygun nodeSelector, taint/toleration, podAffinity/anti-affinity, volume topolojisi, port çakışması — bunlar değerlendirilir.
- Scoring (priority): Aday Node'lar puanlanır (least-requested, balanced-allocation, image-locality vs.); en yüksek puanlı seçilir.
- Karar verdikten sonra Pod tanımındaki spec.nodeName'i set eder ve API server'a yazar.
- Scheduler Pod'u Node'a göndermez; sadece atama yapar. Asıl çalıştırma kubelet'in işidir.

kube-controller-manager (KCM) — En Önemli Bileşen

Tek bir binary içinde çalışan onlarca controller'ın bütünüdür. Her controller bağımsız bir kontrol döngüsü çalıştırır: watch → diff (desired vs current) → act. Hepsi API server'a watch açıp ilgili kaynak tipini izler ve farkı kapatmaya çalışır.

KCM içindeki başlıca controller'lar:

Komut	Açıklama
Deployment Controller	Deployment nesnesini izler; her image/template değişiminde yeni bir ReplicaSet oluşturur, eskisini ölçeklemeye başlayıp yenisini büyütür ve rolling update'i yönetir.
ReplicaSet Controller	Bir RS için "istenilen replika sayısı" ile "mevcut Pod sayısı" arasındaki farkı kapatır. Eksikse Pod yaratır, fazlaysa siler.
Node Controller	Node sağlığını izler. --node-monitor-grace-period (default 40s) içinde kubelet'ten heartbeat gelmezse Node'u NotReady işaretler; --pod-eviction-timeout sonra üzerindeki Pod'ları siler (taint-based eviction).
Endpoints / EndpointSlice Controller	Service'in selector'ına uyan ve Ready olan Pod'ların IP:port listesini günceller. Bu liste kube-proxy'nin iptables/IPVS kurallarını yazması için kaynaktır.
ServiceAccount + Token Controller	Her namespace için default ServiceAccount yaratır; ilgili Pod'lara ServiceAccount token Secret'ını otomatik mount eder.

Namespace Controller	Namespace silindiğinde içindeki tüm kaynakları temizler (finalizer mekanizması).
Job / CronJob Controller	Job için gereken sayıda Pod oluşturur; CronJob zamana göre Job tetikler.
StatefulSet / DaemonSet Controller	StatefulSet sıralı Pod yaratımı + PVC bağlama; DaemonSet her Node'a tam 1 Pod yerleştirir.
HPA Controller	metrics-server'dan okur, hedef metriğe göre Deployment/StatefulSet replicas alanını günceller.
PersistentVolume Controller	PVC'yi uygun PV ile bind eder; provisioner ile çalışıyorsa dinamik PV oluşturur.
Garbage Collector	OwnerReferences zincirini takip eder; örnek: Deployment silinince ona ait RS ve Pod'lar otomatik silinir.

İpucu: Akılda kalsın: scheduler Pod'u Node'a yerleştirir; kubelet Pod'u çalıştırır; geri kalan TÜM otomasyon (Pod'u kim yaratacak, kim canlı tutacak, kim sayacak, kim Service'e bağlayacak) kube-controller-manager içindedir.

cloud-controller-manager (CCM)

Cloud sağlayıcısına özgü controller'lar burada toplanır. Yalnızca bulut cluster'larında vardır (Minikube'de yoktur).

- Node Controller (cloud): cloud API'sinden Node metadata'sını (zone, region, instance-type) çeker, etiketler.
- Route Controller: Pod CIDR'ları için VPC route table'larını günceller.
- Service Controller: type: LoadBalancer Service yaratıldığında cloud sağlayıcısının gerçek LoadBalancer'ını (ELB, GLB) provision eder, EXTERNAL-IP'yi geri yazar.

Worker Node Bileşenleri

Komut	Açıklama
kubelet	Her Node'da bir adet. API server'a sürekli watch açar; kendisine atanan Pod'ları runtime üzerinde yaratır, probe'ları çalıştırır.
kube-proxy	Service'in ClusterIP'sine gelen paketleri uygun Pod IP'sine yönlendiren iptables/IPVS kurallarını yönetir.
Container runtime	containerd, CRI-O ya da Docker (deprecated). CRI arabirimi üzerinden konuşur.

Reconciliation Loop (Kontrol Döngüsü)

Her controller şu kalıbı uygular: watch ile API server'dan ilgili kaynak değişikliklerini dinler, diff alır (desired vs current), ardından act ederek farkı kapatır. Pod silindiğinde ReplicaSet yeni Pod yaratır; Pod tanımı değiştiğinde Deployment yeni ReplicaSet açar.

Bir Pod Yaratılırken Olanlar — İki Senaryo

Pod nasıl yaratılırsa yaratılsın, her bileşen aynı kontrol döngüsü mantığını izler. Aşağıda iki yaygın senaryo karşılaştırmalı verilmiştir.

Senaryo A — Pod doğrudan oluşturuluyor

kubectl apply -f pod.yaml ile çıplak bir Pod yaratıldığında:

- 1. kubectl → kube-apiserver: Manifest API server'a POST edilir.
- 2. kube-apiserver: Authentication → Authorization (RBAC) → Admission Controllers (LimitRanger, ResourceQuota, MutatingWebhook, ValidatingWebhook) sırasıyla geçirilir. ServiceAccount Admission Pod'a default ServiceAccount token Secret'ını mount eder. Sonra etcd'ye yazılır; Pod Pending durumunda durur.
- 3. kube-scheduler: nodeName'i boş olan Pod'u watch ile görür. Filtering + Scoring çalıştırır, en uygun Node'u seçer, spec.nodeName'i set ederek API server'a yazar.
- 4. kubelet (hedef Node'da): kendisine atanmış Pod'u görür; container runtime'a (containerd/CRI-O) image pull + container start çağrısı yapar. Probe'ları yönetir, status'u günceller.
- 5. kube-controller-manager — Endpoints Controller: Pod Ready olduğunda, eğer bu Pod'un label'larına uyan bir Service varsa, Service'in EndpointSlice'ına Pod IP'sini ekler. Kube-proxy bu değişikliği watch eder, iptables/IPVS kurallarını yazar — Pod artık trafik alır.
- 6. (Pod ölürse) Node Controller (KCM içinde): kubelet heartbeat kesilirse Node'u NotReady işaretler; Pod'u Terminating'e geçirir. Çıplak Pod yeniden yaratılmaz — bu yüzden üretimde Deployment kullanılır.

Senaryo B — Pod, Deployment ile oluşturuluyor (üretim)

Üretimde Pod'lar genellikle Deployment üzerinden yaratılır. Akış zinciri uzar:

- 1. kubectl → kube-apiserver: Deployment manifest'i POST.
- 2. kube-apiserver: Admission'dan geçer, Deployment etcd'ye yazılır.
- 3. KCM — Deployment Controller: Yeni Deployment'ı görür; pod-template-hash etiketli yeni bir ReplicaSet yaratır.
- 4. KCM — ReplicaSet Controller: RS'nin istenen replicas sayısını görür; her bir replika için bir Pod nesnesi yaratır ve API server'a yazar. Pod hâlâ nodeName boş (Pending).
- 5. kube-scheduler: her Pod'a uygun Node seçer.
- 6. kubelet: Pod'ları çalıştırır, runtime'a komut verir.
- 7. KCM — Endpoints / EndpointSlice Controller: Ready olan Pod'ları Service'in endpoint listesine ekler.
- 8. (Bir Pod ölürse) KCM — ReplicaSet Controller tekrar devreye girer, eksik replikayı kapatmak için yeni Pod yaratır → self-healing budur.
- 9. (Image güncellenirse) KCM — Deployment Controller yeni RS açar, eskisini ölçek aşağı çekerek rolling update başlatır.

İpucu: Görüldüğü gibi kube-controller-manager iki seviyede aktiftir: (a) Pod yaratımı için Deployment → ReplicaSet zincirini sürmek, (b) Pod yaratıldıktan sonra hayatta tutmak, Service'e bağlamak, Node arızasını yönetmek. Scheduler ve kubelet "tek seferlik" işler yaparken, KCM sürekli çalışan beyin gibidir.

45 Dakikalık Akış Önerisi

0-10 dk	Kontrol döngüsü fikrini anlat; cluster topolojisini çiz.
10-25 dk	Control plane bileşenlerini sırayla işle; her birinin sorumluluğunu örneklerle göster.
25-35 dk	Worker tarafı: kubelet'in API server'a watch açması, kube-proxy'nin iptables kuralları.
35-45 dk	Lab: kubectl get nodes -o wide, kubectl get pods -n kube-system, kubectl logs -n kube-system kube-controller-manager-... — bileşenleri yaşayan halde göster.

Mini Lab — Cluster'ı tanıyın

```
kubectl cluster-info
kubectl get nodes -o wide
kubectl get pods -n kube-system          # control plane Pod'ları
kubectl get pods -n kube-system -l component=kube-controller-manager
kubectl logs -n kube-system -l component=kube-controller-manager --tail=20
kubectl api-resources | head -20
kubectl explain pod.spec --recursive | head -40
```

M2. Pod (120 dk)

1) Amaç ve Mantık

Pod, Kubernetes'te dağıtımın en küçük birimidir. Bir Pod, birlikte çalışması mantıklı olan bir veya birden fazla konteynerden oluşur ve şu kaynakları paylaşır: aynı ağ ad uzayı (Pod IP), aynı IPC ad uzayı, paylaşılan volume'ler.

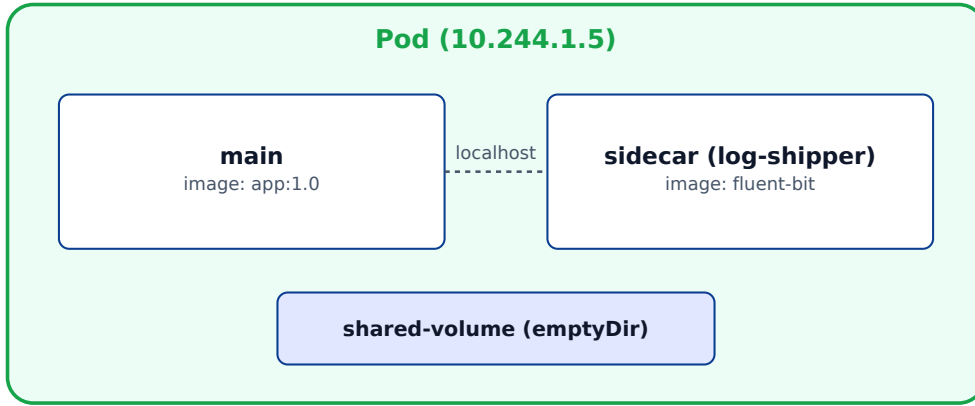
Tek konteynerli Pod en yaygın kullanımdır. Çok konteynerli Pod kalıbına klasik örnek sidecar (log toplayıcı, proxy, config reloader gibi).

Pod, kendi başına yeniden başlatılmaz ve ölçeklenmez. Self-healing isteniyorsa ReplicaSet veya Deployment kullanılmalıdır. Tek başına Pod, ancak özel durumlarda (debug, geçici görev) anlamlıdır.

2) Mimarideki Yeri

Pod, scheduler tarafından bir Node'a yerleştirilir ve o Node'daki kubelet tarafından yaşatılır. Aynı Pod'daki konteynerler her zaman aynı Node'da olur; bunlar başka Pod'lara dağıtılamaz.

Pod Anatomisi — Paylaşılan Ağ ve Volume



Aynı Pod'daki konteynerler:

- Aynı IP'yi paylaşır
- localhost ile birbirini görür
- Volume'leri paylaşır
- Birlikte yaratılır, birlikte ölür

Bu sebeple Pod, dağıtımın en küçük birimi sayılır — konteyner değil.

3) Komutlar ve Önemli Flag'ler

Pod'u imperative yaratmanın en hızlı yolu `kubectl run` komutudur. Üretimde bunun yerine YAML manifest tercih edilir.

Komut	Açıklama
<code>kubectl run nginx --image=nginx:1.27</code>	Tek bir Pod yaratır (eski sürümlerde Deployment yaratırdı; 1.18+ artık sadece Pod).
<code>kubectl get pods -o wide</code>	Pod listesi + IP, Node, durum.

kubectl describe pod <ad>	Events, container status, probe sonuçları, mount detayları.
kubectl logs <ad> [-c <container>] [-f] [--tail=N]	Konteyner stdout/stderr. Çok konteynerli Pod'da -c zorunludur. -f follow (canlı izleme), --tail=N son N satır, --since=10m son 10 dakika, --timestamps her satıra zaman damgası ekler.
kubectl logs <ad> --previous (veya -p)	Bir önceki konteyner instance'ının loglarını gösterir. Konteyner crash olup restart edildiğinde kubectl logs <ad> sadece yeni başlayan instance'ı gösterir — ama yeni instance henüz hata üretmediyse boş çıkar. --previous ile az önce ölen konteynerin son stdout/stderr çıktısını okursunuz; CrashLoopBackOff teşhisinde birinci başvuru komuttur. Yalnızca bir önceki instance'ı saklar; iki önce öleni gösteremez. -c ile çok konteynerli Pod'da hedef konteyneri seçin.
kubectl exec -it <ad> -- sh	Pod içinde shell. -- sonrası komut çalışır.
kubectl port-forward pod/<ad> 8080:80	Lokal makineden Pod'a tünel.
kubectl cp local.txt <ad>:/tmp/	Dosya kopyalama (debug için).
kubectl delete pod <ad> [--grace-period=0 --force]	Pod'u sonlandırır; force ise grace period beklemez.
kubectl get pod <ad> -o yaml	Mevcut tanımı YAML olarak görür (status dahil).
kubectl debug -it <ad> --image=busybox --target=<c>	Ephemeral debug container ekler (K8s 1.25+).

kubectl run için en sık kullanılan flag'ler:

Flag	Anlam	Örnek
--image=	Çalıştırılacak konteyner image'ı.	--image=nginx:1.27
--port=<n>	containerPort tanımlar (Pod IP üzerinden açılır).	--port=8080
--env=KEY=VAL	Ortam değişkeni enjekte eder; birden çok kullanılabilir.	--env=APP_MODE=dev
--labels=k1=v1,k2=v2	Pod'a etiket ekler. Service selector'ları için kritik.	--labels=app=web,tier=front
--restart=Never	Pod'un restartPolicy'sini Never'a set eder. Konteyner exit ettiğinde kubelet konteyneri yeniden başlatmaz; Pod Failed/Succeeded terminal durumuna düşer. Tek seferlik komutlar, batch işler, debug için idealdir. Diğer değerler: Always (default — her exit'te yeniden başlat), OnFailure (yalnız hatalı exit'te yeniden başlat — Job senaryosu).	--restart=Never

--rm	Pod sonlanınca otomatik temizle. --restart=Never ile birlikte kullanılır.	--rm
--command -- <cmd>	Image'ın default ENTRYPOINT/CMD'sini ezer; tüm argümanlar konteynere geçer.	--command -- sleep 3600
--dry-run=client -o yaml	Manifest üretir, uygulamaz. apply etmeden YAML görmek için.	> pod.yaml
-n <namespace>	Hangi namespace'te oluşturulacak.	-n dev
--image-pull-policy	Konteyner image'ının ne zaman çekileceğini belirler: Always — her Pod start'ında registry'den yeniden çek (tag :latest için default). IfNotPresent — image Node'da zaten varsa kullan, yoksa çek (sabit tag için default). Never — asla çekme; image Node'da yoksa Pod ErrImageNeverPull verir. Minikube'de minikube image load ile yüklenen lokal image'ları kullanmak için Never veya IfNotPresent şarttır, yoksa K8s registry'ye uzanır ve hata alır.	--image-pull-policy=IfNotPresent

4) Declarative — YAML Manifest

Yaml manifest hem versiyon kontrolüne girer hem de tekrarlanabilir. spec.containers bir liste olduğu için her Pod birden fazla konteyner barındırabilir.

```

apiVersion: v1
kind: Pod
metadata:
  name: web
  labels:
    app: web
    tier: front
spec:
  restartPolicy: Always      # Always | OnFailure | Never
  terminationGracePeriodSeconds: 30
  containers:
  - name: nginx
    image: nginx:1.27
    imagePullPolicy: IfNotPresent
    ports:
    - name: http
      containerPort: 80
      protocol: TCP
    env:
    - name: APP_MODE
      value: "prod"
    - name: POD_NAME          # Downward API
      valueFrom:
        fieldRef:
          fieldPath: metadata.name
  resources:
    requests:
      cpu: "50m"
      memory: "64Mi"

```

```
limits:
  cpu: "500m"
  memory: "256Mi"
volumeMounts:
- name: cache
  mountPath: /var/cache/nginx
volumes:
- name: cache
  emptyDir: {}
```

5) İleri Detaylar, Best Practice ve Sık Hatalar

- restartPolicy Pod seviyesindedir; Deployment her zaman Always ister. OnFailure/Never yalnızca Job/CronJob senaryolarında anlamlıdır.
- containerPort sadece bilgi amaçlıdır — Pod'un IP'sine gerçek port erişimini engellemez. Asıl önemi Service'in targetPort: http gibi isimle referans verebilmesidir.
- imagePullPolicy: tag :latest ise default Always, sabit tag ise IfNotPresent'tir. Sabit tag kullanın, :latest'tan kaçının.
- Pod yaşam döngüsü: Pending → ContainerCreating → Running → Succeeded/Failed/Terminating. Terminal durumlar yeniden başlatılmaz.
- Init containers sıralı çalışır; tümü tamamlanmadan ana konteynerler başlamaz. Bağımlılık beklemek, şema migrasyonu, dosya hazırlamak için idealdir.
- Ephemeral containers (1.25+) ile çalışan Pod'a debug konteyneri eklenebilir; kubectl debug. Hiçbir Pod yeniden başlatılmaz.
- Pod IP geçicidir: Pod silince yeni Pod yeni IP alır. Bu yüzden başka uygulamalar Pod IP'sine değil, Service'in DNS adına bağlanmalıdır.
- terminationGracePeriodSeconds (default 30): Pod'a SIGTERM atılır, bu süre kadar beklenir, süre dolarsa SIGKILL. preStop hook ile düzgün kapanış senaryosu kurgulanabilir.
- SecurityContext: runAsNonRoot: true, allowPrivilegeEscalation: false, readOnlyRootFilesystem: true — üretim için minimum şart.
- Sık hata: Pod'u doğrudan ölçeklemeye çalışmak. Pod ölçeklenemez — onun yerine ReplicaSet veya Deployment.
- Sık hata: Konteyner ölünce "Pod öldü" sanmak. Aynı Pod içindeki konteyner restart edilebilir; Pod hâlâ aynı IP'dedir. kubectl get pod RESTARTS sütununu izleyin.
- Image olmadan başarısız Pod: Status ErrImagePull veya ImagePullBackOff. Çözüm: image tag'i, registry erişimi ve imagePullSecrets'i kontrol edin.

Lab — Pod ile yapılacaklar

```
# 1) Tek konteynerli Pod
kubectl run web --image=nginx:1.27 --port=80 --labels=app=web
kubectl get pods -o wide
kubectl describe pod web | head -40
kubectl port-forward pod/web 8080:80 &
curl http://localhost:8080

# 2) İki konteynerli Pod (sidecar)
kubectl apply -f sidecar-pod.yaml
kubectl logs sidecar -c app
kubectl logs sidecar -c log-shipper

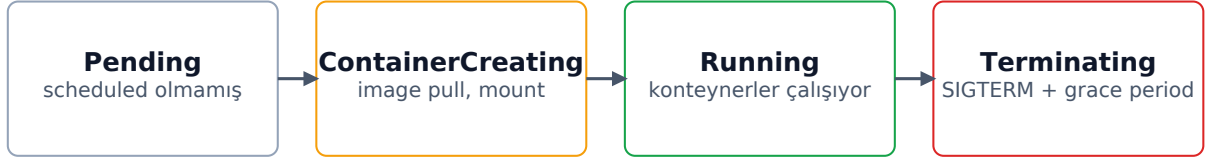
# 3) Ephemeral debug
```

```
kubectl debug -it web --image=busybox --target=web -- sh
```

4) Pod yaşam döngüsünü gör

```
kubectl delete pod web --grace-period=0 --force
```

Pod Yaşam Döngüsü (4 evre)



kubectl get pod STATUS sütunu burada görünür. Failed/Succeeded terminal durumlarıdır.

M3. ReplicaSet ve Deployment (120 dk)

1) Amaç ve Mantık

ReplicaSet belirli sayıda Pod'un her zaman çalışır kalmasını sağlar. Selector'a göre eşleşen Pod sayısını sayar; eksikse yeni Pod yaratır, fazla ise siler. Self-healing budur.

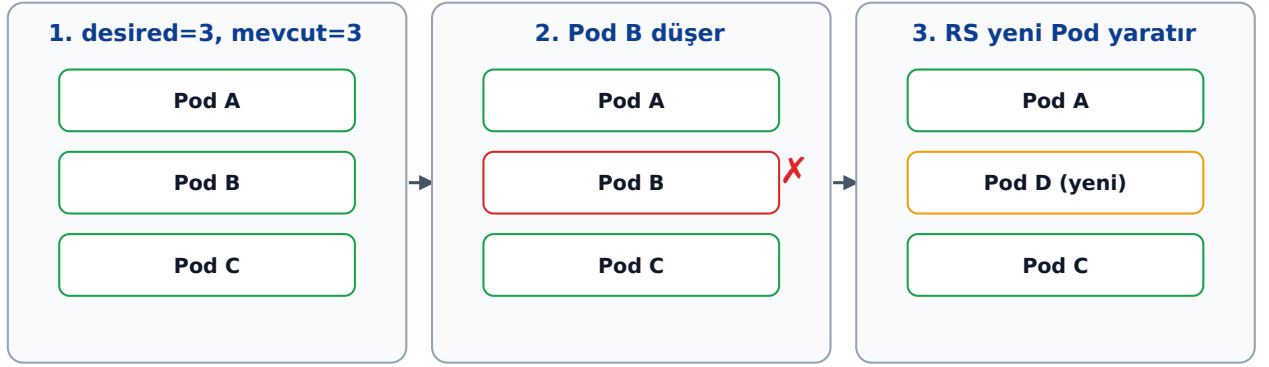
Deployment, ReplicaSet'in üzerine rollout, rollback ve history ekler. Bir Deployment her image değişikliğinde yeni bir ReplicaSet yaratır; trafiği kademeli olarak eski RS'den yeni RS'e kaydırır.

Üretim ortamında doğrudan ReplicaSet yazılmaz — her zaman Deployment ile yönetilir.

2) Mimarideki Yeri

Deployment → yeni bir ReplicaSet → bu RS, label selector ile kendi Pod'larını sahiplenir. Bu zincir kurslarda en çok kafa karıştıran konudur ve dikkatle anlatılmalıdır.

ReplicaSet — Self-Healing (3 kare)



ReplicaSet sürekli olarak istenen replika sayısını kontrol eder ve farkı kapatır.

3) Komutlar ve Önemli Flag'ler

Aşağıdaki komutlar gündelik kullanımın %90'ını kapsar:

Komut	Açıklama
<code>kubectl create deployment web --image=nginx:1.27 --replicas=3</code>	Hızlı oluşturma (imperative). Manifest üretmek için <code>--dry-run=client -o yaml</code> .
<code>kubectl get deploy,rs,pods -l app=web</code>	Deployment → RS → Pod zincirini birlikte görür.
<code>kubectl scale deploy/web --replicas=5</code>	Replika sayısını anlık değiştirir.
<code>kubectl set image deploy/web nginx=nginx:1.28</code>	Konteynerin image'ını günceller; yeni rollout tetikler.
<code>kubectl rollout status deploy/web</code>	Rollout'un ilerleyişini gerçek zamanlı izler.
<code>kubectl rollout history deploy/web</code>	Tüm revizyonları listeler.

kubectl rollout undo deploy/web [--to-revision=N]	Önceki sürüme döner; isteğe bağlı belirli revizyona.
kubectl rollout pause/resume deploy/web	Devam eden rollout'u duraklatır/devam ettirir.
kubectl rollout restart deploy/web	Aynı image'la yeni rollout — env/ConfigMap değişimi sonrası kullanılır.
kubectl edit deploy/web	Tanımı editörle aç; kaydedince diff uygulanır.

Deployment'a özgü kritik alanlar (YAML'da kullanılan ama komutta --set/-p ile de düzeltilebilen):

Flag	Anlam	Örnek
strategy.type	RollingUpdate (default) veya Recreate (önce tamamı durur).	Recreate
rollingUpdate.maxSurge	Hedef replikanın üzerine ekstra kaç Pod açılabilir.	25% veya 1
rollingUpdate.maxUnavailable	Aynı anda kaç Pod hazır olmayabilir.	25% veya 0
revisionHistoryLimit	Saklanan eski RS sayısı (rollback için).	10 (default)
progressDeadlineSeconds	Rollout bu süre içinde ilerlemezse fail sayılır.	600 (default)
minReadySeconds	Bir Pod hazır sayıldıktan sonra bekleme süresi.	10
--record	(eski) kubectl komutunu annotation'a yazar.	kubectl set image ... --record

4) Declarative — YAML Manifest

Aşağıdaki manifest 'altın standart' bir Deployment örneğidir:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels: { app: web }
spec:
  replicas: 3
  revisionHistoryLimit: 5
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0    # her zaman tüm replikalar hazır
  selector:
    matchLabels: { app: web }
  template:
    # bu kısım Pod template'i
    metadata:
      labels: { app: web } # selector ile birebir uyumlu OLMALI
    spec:
      containers:
        - name: web

```

```
image: nginx:1.27
ports: [{ name: http, containerPort: 80 }]
readinessProbe:
  httpGet: { path: /, port: http }
  periodSeconds: 5
resources:
  requests: { cpu: 50m, memory: 64Mi }
  limits: { cpu: 500m, memory: 256Mi }
```

5) İleri Detaylar, Best Practice ve Sık Hatalar

- selector immutable'dır: bir kere ayarlanan spec.selector sonradan değiştirilemez (yeni Deployment oluşturmak gerekir).
- Template label'ları selector'la birebir uyumlu olmak zorunda. Olmazsa kubectl apply hata verir.
- maxUnavailable: 0 + readinessProbe = sıfır downtime rollout. Readiness probe yoksa Pod "hazır değil" sinyalini veremez ve trafik kesilebilir.
- Pause/Resume: birden fazla değişikliği tek seferde uygulamak için rollout'u pause ile durdurun, değişiklikleri yapın, resume ile başlatın.
- Rollback hızlıdır: önceki RS hâlâ duruyor olabilir (revisionHistoryLimit). Bu yüzden bu değeri çok düşük tutmayın.
- Pod template hash etiketi: K8s her RS'e pod-template-hash ekler. Bu sayede selector'lar çakışmaz; bu etikete manuel müdahale etmeyin.
- Recreate stratejisi StatefulSet/DB benzeri uygulamalar için bazen tercih edilir; downtime yaşar ama veri tutarlılığı korur.
- Sık hata: kubectl apply sonrası rollout başlamıyorsa, muhtemelen sadece etiket veya annotation değişmiştir; image değişmediğinde RS yeniden oluşmaz.
- Sık hata: Yanlış selector. Eski Pod'lar Deployment tarafından sahiplenilirse (çakışan label) beklenmedik replikalar oluşur.
- HPA ile etkileşim: Deployment'a HPA bağlıysa spec.replicas alanını manifest'ten kaldırın (yoksa apply her seferinde geri çeker).

Rolling Update Akışı — görsel

Deployment — Rolling Update Akışı (maxSurge=1, maxUnavailable=0)



★ = bu adımda eklenen Pod. Trafik kesintisiz kalır çünkü maxUnavailable=0.

Lab — Rolling update + rollback senaryosu

1) Deployment kur

```
kubectl create deployment web --image=nginx:1.27 --replicas=3
```

```
kubectl rollout status deploy/web
```

2) Yeni image yayınla, kaydı izle

```
kubectl set image deploy/web nginx=nginx:1.28
```

```
watch 'kubectl get pods -l app=web'
```

3) Bilerek hatalı image yayınla

```
kubectl set image deploy/web nginx=nginx:hatali
```

```
kubectl rollout status deploy/web --timeout=30s
```

```
kubectl rollout undo deploy/web
```

4) Replika sayısını HPA gibi değiştir

```
kubectl scale deploy/web --replicas=6
```


M4. Service ve Ağ Modeli (120 dk)

1) Amaç ve Mantık

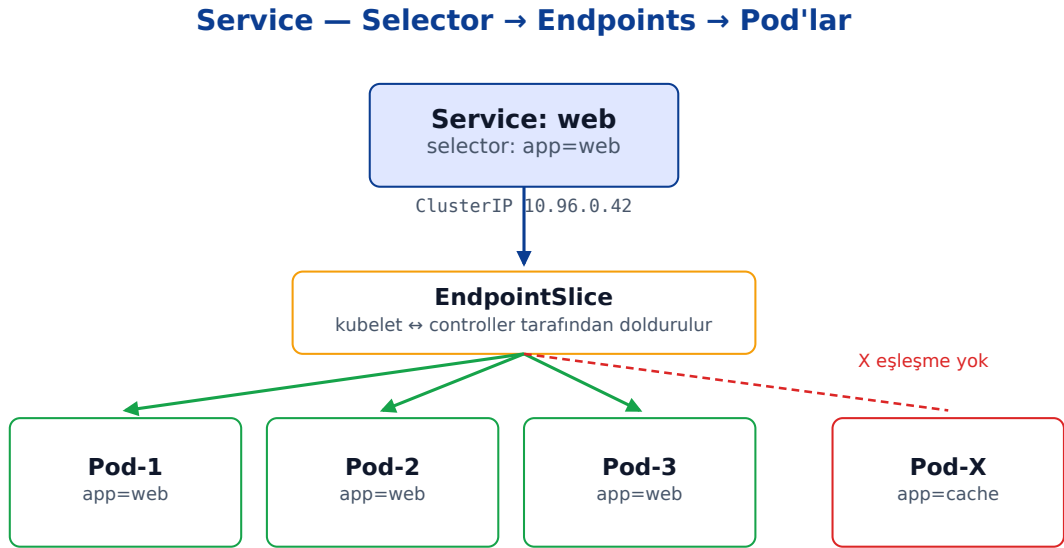
Pod IP'leri geçicidir; Pod silinince yeni Pod yeni IP alır. Bu yüzden başka uygulamalar Pod IP'sine değil Service adıyla bağlanır. Service, label selector ile eşleşen Pod'lara kararlı bir sanal IP (ClusterIP) ve DNS adı sağlar.

Service, gelen trafiği selector ile bulunduğu Pod'lara dağıtır. Dağıtım, kube-proxy'nin Node'lara yazdığı iptables/IPVS kuralları üzerinden L4 seviyesinde yapılır (TCP/UDP, default round-robin).

Service türleri farklı erişim ihtiyaçları için vardır: cluster içi (ClusterIP), Node üstünden (NodePort), bulut LB ile (LoadBalancer) ya da harici DNS yönlendirmesi (ExternalName).

2) Mimarideki Yeri

Service tanımı API server'da saklanır; Endpoints / EndpointSlice controller, selector'a uyan Pod'ları izleyip endpoint listesini günceller. Her Node'daki kube-proxy bu listeyi okuyup iptables/IPVS kurallarını yazar.



Service, Pod IP'lerini değil; selector'la eşleşen Pod'ları endpoints listesinde takip eder.

3) Komutlar ve Önemli Flag'ler

Service'in en yaygın komutları:

Komut	Açıklama
kubectl expose deploy/web --port=80 --target-port=80	Hızlıca ClusterIP Service yaratır.
kubectl expose deploy/web --port=80 --type=NodePort	NodePort olarak açar; port otomatik atanır.
kubectl get svc	Service'leri listeler; CLUSTER-IP, EXTERNAL-IP, PORT(S) sütunlarına bakın.

<code>kubectl get endpoints <ad> -o wide</code>	Service'in mevcut endpoint listesini (Pod IP:port) gösterir.
<code>kubectl get endpointslices</code>	Modern EndpointSlice nesneleri (1.21+ default).
<code>kubectl describe svc <ad></code>	Selector, ports, endpoints, sessionAffinity ayrıntıları.
<code>kubectl port-forward svc/<ad> 8080:80</code>	Lokal makineden Service'e tünel.
<code>kubectl run probe --rm -it --image=busybox -- wget -q0-web.default.svc.cluster.local</code>	Cluster içi DNS ile Service çağırma testi.
<code>kubectl patch svc <ad> -p '{"spec":{"type":"NodePort"}}'</code>	Mevcut Service'in tipini değiştirir.

kubectl expose ve Service tanımında öne çıkan flag'ler:

Flag	Anlam	Örnek
<code>--port</code>	Service'in açtığı port (cluster içinden).	<code>--port=80</code>
<code>--target-port</code>	Pod üzerindeki containerPort veya ismi.	<code>--target-port=http</code>
<code>--type</code>	ClusterIP NodePort LoadBalancer ExternalName.	<code>--type=NodePort</code>
<code>--protocol</code>	TCP (default) UDP SCTP.	<code>--protocol=UDP</code>
<code>--selector</code>	Eşleşecek Pod label'larını override eder.	<code>--selector=app=web</code>
<code>--cluster-ip=None</code>	Headless Service (DNS A kayıtları her Pod için).	(StatefulSet için yaygın)
<code>nodePort</code>	NodePort manuel set; 30000-32767 aralığı.	30080
<code>sessionAffinity</code>	ClientIP olursa aynı IP aynı Pod'a yapışır.	ClientIP
<code>externalTrafficPolicy</code>	Local: trafiği aynı Node'daki Pod'a tut, kaynak IP korunur.	Local

4) Declarative — YAML Manifest

ClusterIP + NodePort + Service ports için isimli targetPort kullanımı:

```

apiVersion: v1
kind: Service
metadata:
  name: web
  labels: { app: web }
spec:
  type: ClusterIP          # default; NodePort/LoadBalancer da olabilir
  selector: { app: web }   # Deployment.spec.template.labels ile uyumlu
  ports:
    - name: http
      port: 80              # cluster içinden bu port
      targetPort: http      # Pod'daki containerPort adı veya numarası
      protocol: TCP
      sessionAffinity: None # ClientIP da yapılabilir
  ---

```

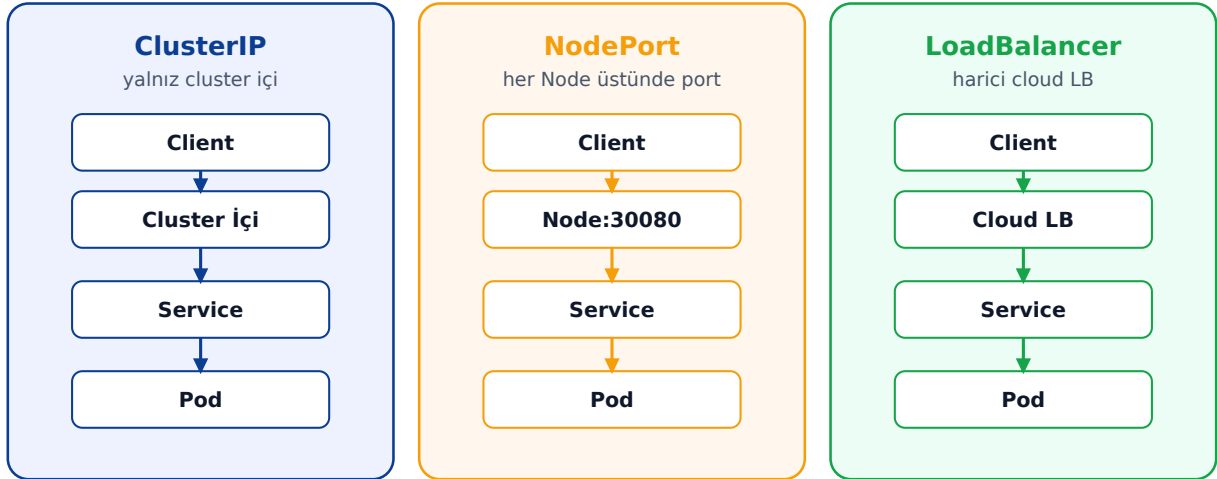
```
# Aynı Pod'lara ikinci bir NodePort Service
apiVersion: v1
kind: Service
metadata:
  name: web-np
spec:
  type: NodePort
  selector: { app: web }
  ports:
    - { port: 80, targetPort: http, nodePort: 30080 }
```

5) İleri Detaylar, Best Practice ve Sık Hatalar

- DNS adı kalıbı: <service>.<namespace>.svc.cluster.local. Aynı namespace içinden sadece web de yeterlidir.
- Headless Service (clusterIP: None) StatefulSet ile birlikte kullanılır; her Pod için ayrı DNS kaydı döner (pod-0.app, pod-1.app gibi).
- EndpointSlice vs Endpoints: Endpoint nesnesi büyük cluster'larda performans sorunu yaşıyordu; 1.21+ EndpointSlice default oldu (büyük listeyi parçalara böler).
- ExternalName Service: selector yok, DNS CNAME yönlendirmesi (örn. db.prod.example.com).
- sessionAffinity: ClientIP sticky session sağlar; cookie tabanlı sticky değildir.
- externalTrafficPolicy: Local NodePort/LB için kaynak IP'yi korur, ama o Node'da Pod yoksa istek düşer.
- Sık hata: Service yaratıldı ama endpoint listesi boş. Sebep: selector yanlış, Pod label'ları uyuşmuyor ya da Pod henüz Ready değil (readiness probe fail).
- Sık hata: targetPort'u image'ın gerçekten dinlediği port ile karıştırmak. Container 8080 dinlerken targetPort 80 ise istek zaman aşımı yer.
- kube-proxy mod: iptables (default) vs IPVS. Büyük cluster'larda IPVS daha performanslı.
- NetworkPolicy (ileri): Service'ler default olarak her Pod'dan erişilebilir. Cluster'da Calico/Cilium gibi NetworkPolicy destekli CNI varsa, trafik filtrelenebilir.
- İpucu — debug: kubectl run probe --rm -it --image=nicolaka/netshoot ile dig, curl, tcpdump, nslookup gibi araçlara erişin.

Service Türleri — Karşılaştırma

Service Türleri — Erişim Yolu Karşılaştırması



LoadBalancer içerir NodePort'u; NodePort içerir ClusterIP'yi.

Lab — Service ile ulaşılabilirlik

```
kubectl create deployment web --image=nginx:1.27 --replicas=3
kubectl expose deploy/web --port=80 --type=ClusterIP
kubectl get endpoints web -o wide
```

```
# Cluster içi DNS testi
kubectl run probe --rm -it --image=busybox -- sh
> wget -q0- web.default.svc.cluster.local
> nslookup web
```

```
# NodePort'a yükselt ve dışarıdan eriş
kubectl patch svc web -p '{"spec":{"type":"NodePort"}}'
minikube service web --url
```

M5. ConfigMap ve Secret (90 dk)

1) Amaç ve Mantık

12-factor uygulama prensibi gereği yapılandırma image'tan ayrı olmalıdır. ConfigMap düz metin yapılandırmayı, Secret ise hassas verileri (parola, token, sertifika) Pod'lara taşımak için kullanılır.

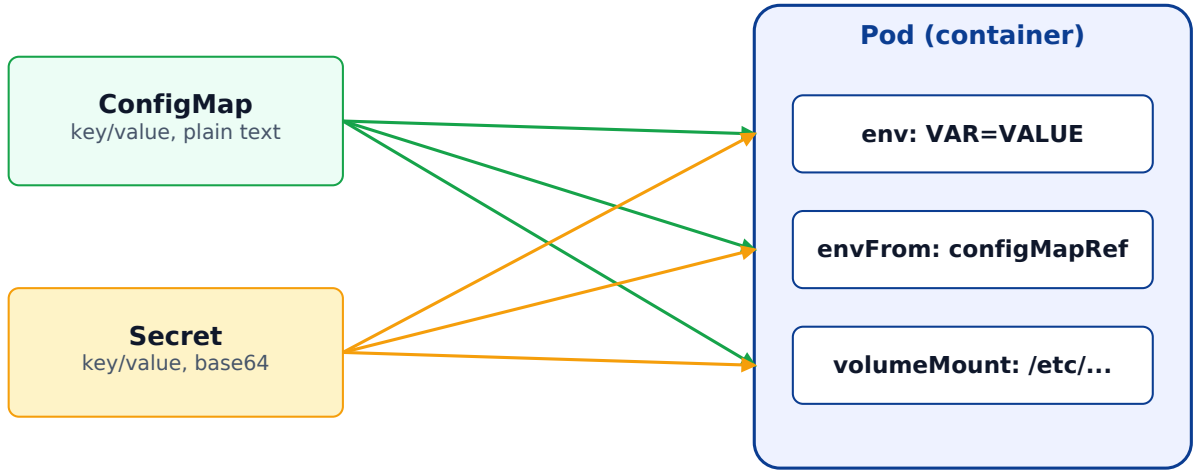
Her ikisi de key/value yapısındadır. Tek fark: Secret değerleri base64 ile kodlanmıştır ve etcd'de (varsayılan olarak) şifrelenmemiş tutulur. Bu yüzden Secret ≠ "şifreli"; encryption at rest ayrıca açılmalıdır.

ConfigMap/Secret üç şekilde Pod'a aktarılır: tek tek env değişkeni (env.valueFrom), toplu env (envFrom), dosya olarak volume mount.

2) Mimarideki Yeri

Her iki nesne de namespace seviyesindedir. Volume olarak mount edildiklerinde ConfigMap güncellendiğinde Pod yeniden başlatılmadan yaklaşık 30-60 sn içinde dosya içeriği tazelenir. Env olarak enjekte edilen değerler tazelenmez — Pod restart gerekir.

ConfigMap & Secret → Pod'a Aktarım Yolları



Volume mount kullanıldığında ConfigMap/Secret güncellemeleri Pod yeniden başlatılmadan tazelenir (≈30-60 sn).

3) Komutlar ve Önemli Flag'ler

Hızlı oluşturma yolları:

Komut	Açıklama
kubectl create configmap app --from-literal=KEY=val1 --from-literal=K2=val2	Literal değerlerden ConfigMap.
kubectl create configmap app --from-file=./config.json	Tek dosyadan ConfigMap; key = dosya adı, value = içerik.
kubectl create configmap app --from-file=cfg/	Dizindeki tüm dosyalar ayrı key olur.

kubectl create configmap app --from-env-file=.env	.env formatında çoklu key/value.
kubectl create secret generic db --from-literal=PASSWORD='S3cret!'	Opaque tipinde generic Secret.
kubectl create secret docker-registry regcred --docker-server=... --docker-username=... --docker-password=...	Private registry için imagePullSecrets.
kubectl create secret tls web-tls --cert=cert.pem --key=key.pem	TLS sertifikası Secret'ı (Ingress için).
kubectl get cm,secret	Listele.
kubectl get secret db -o jsonpath='{.data.PASSWORD}' base64 -d	Secret değerini decode ederek görür.
kubectl edit cm app	Canlı olarak düzenle; volume mount edilmişse içerik tazelenir.

kubectl create configmap/secret için en sık kullanılan flag'ler:

Flag	Anlam	Örnek
--from-literal	key=value şeklinde tek bir giriş.	--from-literal=APP_MODE=prod
--from-file	Dosya veya dizinden.	--from-file=app.conf
--from-file=name=path	Custom key adı ile dosyadan.	--from-file=conf=./prod.json
--from-env-file	.env dosyası formatından.	--from-env-file=.env
--type	Secret için: generic docker-registry tls.	--type=tls
--dry-run=client -o yaml	Manifest üretir, uygulamaz.	> cm.yaml
-n <namespace>	Namespace.	-n dev

4) Declarative — YAML Manifest

ConfigMap + Secret + Pod entegrasyonu örneği:

```

apiVersion: v1
kind: ConfigMap
metadata: { name: app-config }
data:
  APP_MODE: prod
  LOG_LEVEL: info
  appsettings.json: |
    { "feature": true }
---
apiVersion: v1

```

```

kind: Secret
metadata: { name: app-secret }
type: Opaque
stringData:          # base64'e otomatik dönüştürülür
  DB_PASSWORD: 'S3cret!'
---
apiVersion: v1
kind: Pod
metadata: { name: app }
spec:
  containers:
  - name: app
    image: myapp:1.0
    envFrom:          # tüm ConfigMap key'leri env olur
    - configMapRef: { name: app-config }
    - secretRef:     { name: app-secret }
    env:              # tek bir Secret değerini özel adla
    - name: DB_PWD
      valueFrom:
        secretKeyRef:
          name: app-secret
          key: DB_PASSWORD
    volumeMounts:
    - { name: cfg, mountPath: /etc/app, readOnly: true }
  volumes:
  - name: cfg
    configMap:
      name: app-config
      items:
      - { key: appsettings.json, path: appsettings.json }

```

5) İleri Detaylar, Best Practice ve Sık Hatalar

- Volume mount canlıdır: ConfigMap güncellenince Pod yeniden başlamadan dosya tazelenir. Ama yalnızca subPath kullanılmadığında. subPath ile mount ettiyseniz tazelenmez.
- Env değişkenleri statik: ConfigMap güncellense bile env değişmez. Genelde kubectl rollout restart deploy/... gerekir.
- Immutable ConfigMap/Secret: immutable: true ile değişmez yapılır; API server'ın watch yükünü azaltır; üretim için önerilir.
- Secret base64 ≠ şifrelidir. Etcd encryption at rest mutlaka açılmalı; KMS entegrasyonu önerilir.
- İçerik boyutu: Her ConfigMap/Secret max 1 MB. Büyük dosyalar için PVC veya init container ile indirme yapılır.
- imagePullSecrets: Private registry için Pod.spec.imagePullSecrets[].name kullanılır.
- External Secrets: Vault, AWS Secrets Manager, GCP Secret Manager gibi kaynaklarla entegrasyon için External Secrets Operator (üçüncü taraf) yaygındır.
- Sık hata: Secret'ı git'e commit etmek. Hatta kubectl create secret ... --dry-run -o yaml çıktısı bile base64'lü olduğu için git'e koymaktan kaçının. SealedSecrets/SOPS kullanın.
- Sık hata: ConfigMap'i sadece dosya olarak mount edip ama env'le karıştırmak. Mount edilen dosyada değişiklik var ama env'de yok — uygulama hangi kaynağı okuyor netleştirin.

M6. Volume, PersistentVolume ve PersistentVolumeClaim (90 dk)

1) Amaç ve Mantık

Konteyner dosya sistemi geçicidir; konteyner restart olunca kaybolur. Veri saklamak için Kubernetes'te Volume kavramı vardır. Volume Pod yaşam süresine bağlıdır (Pod silinince çoğu volume kaybolur).

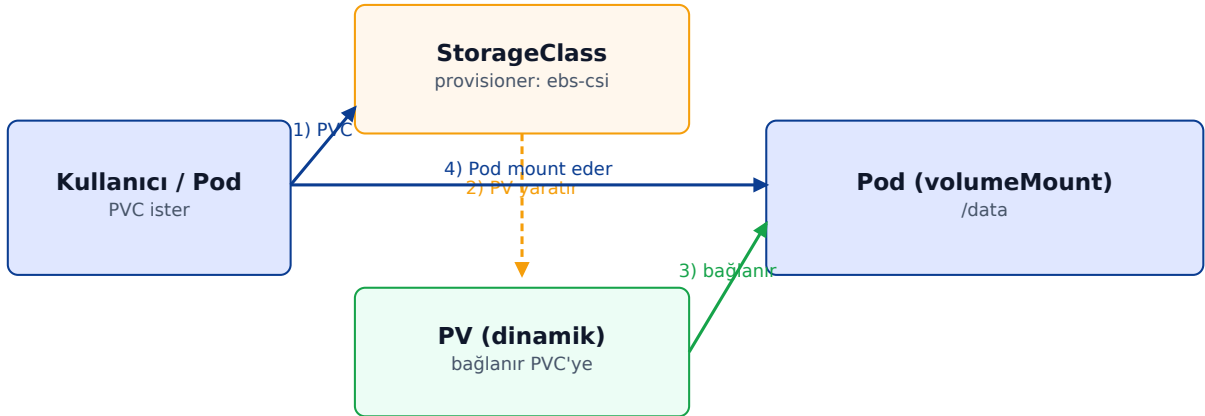
Pod ömrünü aşan kalıcılık için PersistentVolume (PV) ve PersistentVolumeClaim (PVC) kullanılır. PV admin tarafından ya da StorageClass üzerinden dinamik provision edilen gerçek depolama kaynağıdır; PVC kullanıcının bu kaynaktan talebidir.

Pod, doğrudan PV'ye değil PVC üzerinden bağlanır. Bu sayede uygulama kodu altyapıdan bağımsız kalır.

2) Mimarideki Yeri

Volume tipleri: emptyDir (Pod ömrüyle), hostPath (Node disk; üretimde tehlikeli), configMap ve secret (daha önce işlendi), persistentVolumeClaim (PVC referansı), ve cloud-specific (awsElasticBlockStore, gcePersistentDisk) — günümüzde tamamı CSI sürücülerini üzerinden çalışır.

PVC → StorageClass → PV → Pod (Dinamik Provisioning)



PVC sadece istektir. Manuel PV de yapılabilir ama prodüksiyonda StorageClass ile dinamik daha yaygındır.

3) Komutlar ve Önemli Flag'ler

PV/PVC ile gündelik kullanım:

Komut	Açıklama
kubectl get sc,pv,pvc	StorageClass, PV ve PVC'leri birlikte gör.
kubectl describe pvc <ad>	Events bölümünde provisioning hataları görünür.
kubectl get sc	Mevcut StorageClass'lar; (default) işaretli olan otomatik kullanılır.

kubectl apply -f pvc.yaml	PVC oluştur; storageClassName uygun ise dinamik PV açılır.
kubectl delete pvc <ad>	PVC sil. PV'nin reclaim politikasına göre PV ya silinir ya kalır.
kubectl patch pv <ad> -p '{"spec":{"persistentVolumeReclaimPolicy":"Retain"}}'	Reclaim politikasını manuel değiştir.
kubectl get pv -o wide	Capacity, accessModes, claim sahibi, status.
minikube ssh -- ls /var/lib/...	Minikube'de PV'nin fiziksel konumunu kontrol et.

PVC manifest'inde dikkat edilecek alanlar:

Flag	Anlam	Örnek
storageClassName	Hangi SC ile provision edilsin; boş bırakırsa default SC kullanılır.	standard
accessModes	ReadWriteOnce ReadOnlyMany ReadWriteMany ReadWriteOncePod.	["ReadWriteOnce"]
resources.requests.storage	İstenen kapasite.	5Gi
volumeMode	Filesystem (default) veya Block.	Filesystem
dataSource	Snapshot/clone'dan başlat.	VolumeSnapshot
reclaimPolicy	PV silindiğinde: Retain Delete Recycle (deprecated).	Retain
volumeBindingMode	Immediate vs WaitForFirstConsumer (Pod scheduling sonrası).	WaitForFirstConsumer

4) Declarative — YAML Manifest

Tipik PVC + Deployment mount:

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: data }
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 5Gi
  storageClassName: standard
---
apiVersion: apps/v1
kind: Deployment
metadata: { name: app }
spec:
  replicas: 1 # RWX değilse 1 olmalı
  selector: { matchLabels: { app: app } }
  template:
    metadata: { labels: { app: app } }
    spec:
      containers:

```

```
- name: app
  image: myapp:1.0
  volumeMounts:
    - { name: data, mountPath: /data }
volumes:
  - name: data
    persistentVolumeClaim:
      claimName: data
```

5) İleri Detaylar, Best Practice ve Sık Hatalar

- accessModes'lar PVC seviyesindedir ama gerçek davranış storage'a bağlıdır. EBS ve GCE PD RWO destekler; NFS, Azure Files RWX destekler. Yanlış seçim provisioning'i engeller.
- ReadWriteOnce bir Node'a bağlanır (ReadWriteOncePod 1.22+ ile bir Pod'a). Bu yüzden RWO ile replicas>1 Deployment çakışır.
- StatefulSet her replika için ayrı PVC yaratır (volumeClaimTemplates). Veritabanları için tipik yapıdır.
- VolumeBindingMode: WaitForFirstConsumer: PVC, ilgili Pod scheduler'a düşene kadar PV bağlanmaz. Topology-aware storage için kritiktir.
- reclaimPolicy: Retain üretim verisi için güvenli default. Delete politikası, PVC silindiğinde veri kaybetmeye neden olabilir.
- Snapshot ve clone: CSI sürücüleri VolumeSnapshot ile snapshot, dataSource ile clone destekler.
- hostPath kullanmayın (debug hariç): Pod'un yeniden schedule olduğu Node değişirse veri kaybolur.
- Sık hata: PVC "Pending" — sebep çoğunlukla: SC yok, accessMode storage tarafından desteklenmiyor, kapasite kalmadı. kubectl describe pvc ile Events okuyun.
- Sık hata: subPath kullanıldığında ConfigMap canlı tazeleme çalışmaz.
- İpucu — Minikube: default standard SC vardır (hostPath provisioner). Bu yüzden çoklu Node simülasyonunda PVC'ler sınırlı davranır.

M7. Namespace, Label, Annotation (45 dk)

1) Amaç ve Mantık

Namespace, aynı cluster içinde mantıksal izolasyon sağlar. Aynı isimli iki Pod aynı namespace'te olamaz, ama farklı namespace'lerde olabilir. RBAC, ResourceQuota, NetworkPolicy gibi politikalar namespace scope'una kapsanır.

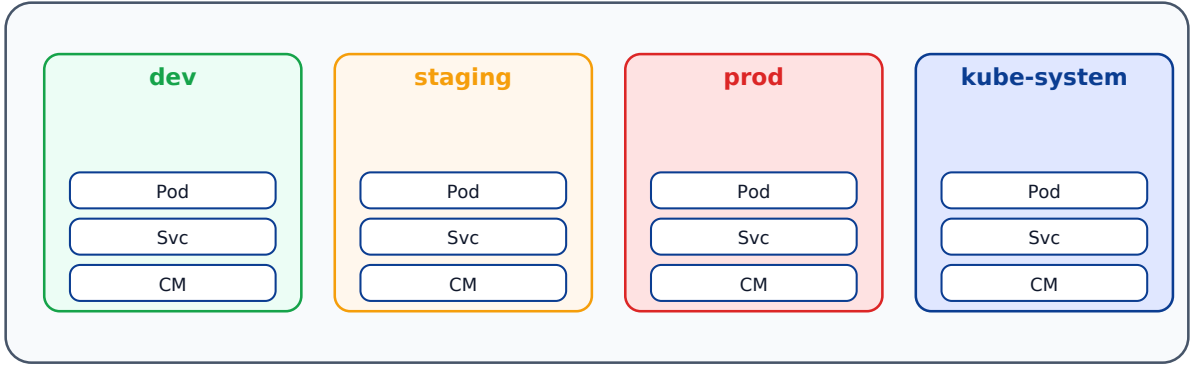
Label, key=value şeklinde nesnelere yapışan ve selector'lar tarafından kullanılan etiketlerdir. Service, ReplicaSet, NetworkPolicy, NodeSelector hep label seçer.

Annotation, label gibi key=value taşır ama selector ile kullanılmaz. Daha çok araç/sistem meta verisi içindir: build info, deployment timestamp, ingress controller'a hint.

2) Mimarideki Yeri

Namespace cluster içinde bir scope'tur — Node'lar paylaşılır, kaynaklar paylaşılır; izolasyon mantıksaldır. kube-system control plane bileşenleri için; default, kube-public, kube-node-lease built-in'lerdir.

Namespace — Sanal Cluster İçinde İzolasyon Cluster (paylaşılan API + Node havuzu)



Namespace'ler kaynak isimleri için scope sağlar; Node'lar paylaşılır, izolasyon mantıksaldır.

3) Komutlar ve Önemli Flag'ler

Namespace ve label yönetiminin temel komutları:

Komut	Açıklama
kubectl create namespace dev	Yeni namespace yaratır.
kubectl get all -n dev	Belirli namespace'teki tüm temel kaynaklar.
kubectl get all -A head	Tüm namespace'lerden listeler.
kubectl config set-context --current --namespace=dev	Geçerli context'in default namespace'ini değiştirir.
kubectl label pod web env=prod tier=front	Bir Pod'a iki label ekler.
kubectl label pod web env-	env label'ını siler (eksi işareti).
kubectl get pods -l app=web,env=prod	Çoklu label selector.

kubectl get pods -l 'env in (prod, staging)'	Set-based selector.
kubectl annotate deploy/web owner=team-x build=abc123	Annotation ekler.
kubectl get pods --show-labels	Pod'ların tüm label'larını gösterir.

Label/annotation komutlarında öne çıkan flag'ler:

Flag	Anlam	Örnek
--overwrite	Mevcut label/annotation'ı değiştirmek için zorunlu.	--overwrite
-l, --selector	Equality (=, !=) ve set-based (in, notin, exists).	-l 'env in (prod)'
--all-namespaces, -A	Tüm namespace'lerden.	kubectl get pods -A
-o jsonpath=...	Tek değer çekmek için.	{.items[0].metadata.labels.app}
--field-selector	metadata.name, status.phase gibi alanlara göre.	status.phase=Running

4) Declarative — YAML Manifest

Namespace + ResourceQuota + LimitRange üçlüsü tipik bir kurulumdur:

```

apiVersion: v1
kind: Namespace
metadata:
  name: dev
  labels: { env: dev, team: backend }
---
apiVersion: v1
kind: ResourceQuota
metadata: { name: dev-quota, namespace: dev }
spec:
  hard:
    pods: "20"
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
---
apiVersion: v1
kind: LimitRange
metadata: { name: dev-limits, namespace: dev }
spec:
  limits:
    - type: Container
      default: { cpu: 200m, memory: 256Mi }
      defaultRequest: { cpu: 50m, memory: 64Mi }
```

5) İleri Detaylar, Best Practice ve Sık Hatalar

- Namespace silmek içindeki tüm kaynakları siler — geri alınamaz. Önce kubectl get all -n <ns> ile kontrol edin.

- Cross-namespace çağrı: `http://web.dev.svc.cluster.local`. NetworkPolicy ile sınırlanabilir.
- Label kuralları: key max 63 karakter, prefix isteğe bağlı (örn. `app.kubernetes.io/name`); değer max 63 karakter, alfa-sayısal/`-/_` vb.
- Önerilen standart label'lar: `app.kubernetes.io/name`, `/instance`, `/version`, `/component`, `/part-of`, `/managed-by`.
- Annotation kullanım örnekleri: `kubernetes.io/change-cause` (rollout history mesajı), `nginx.ingress.kubernetes.io/rewrite-target`.
- ResourceQuota namespace toplam kaynak limitidir; ihlal edilirse yeni Pod yaratılamaz.
- LimitRange namespace içinde default request/limit verir; resources tanımı eksik Pod'lar yine de güvenli çalışır.
- Sık hata: Default namespace'i unutmak. `kubens` (`kubectx` aracı) ile namespace değiştirmek pratiktir.
- Sık hata: Selector'da değişiklik yapmaya çalışmak. `spec.selector` birçok nesnede immutable'dir.

M8. Resource Yönetimi ve Probe'lar (75 dk)

1) Amaç ve Mantık

resources.requests: Pod'un çalışmaya başlaması için Node üzerinde ayrılması istenen CPU/RAM. Scheduler bu değere bakarak Pod'u Node'a yerleştirir. Garanti edilen alt sınırdır.

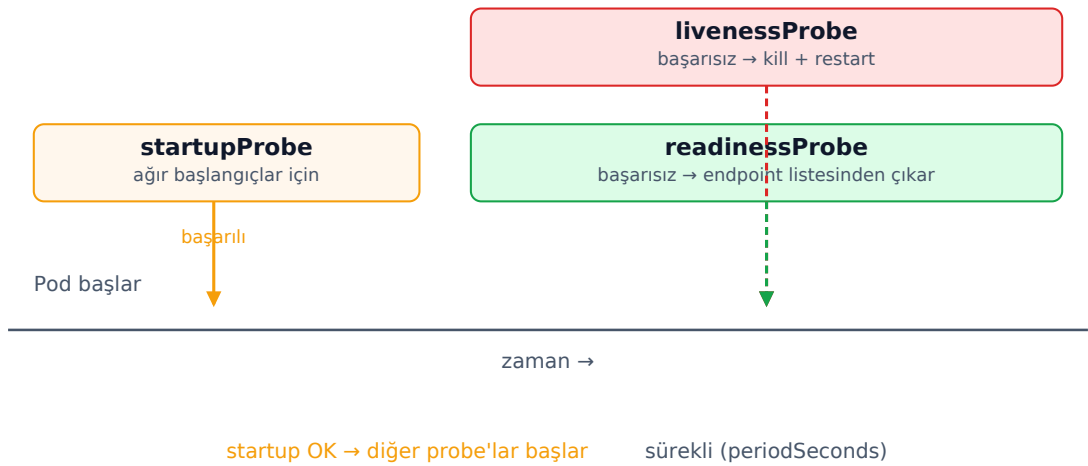
resources.limits: Pod'un en fazla kullanabileceği değerler. CPU'da throttle, bellekte OOMKilled'a sebep olur. Limit yoksa bir Pod Node'un tamamını tüketebilir.

Probe'lar uygulamanın gerçekten çalışıp çalışmadığını anlamak için vardır: **livenessProbe** (canlı mı), **readinessProbe** (trafik almaya hazır mı), **startupProbe** (uzun başlangıçlı uygulamalar için).

2) Mimarideki Yeri

Scheduler requests'e bakar, Pod'u uygun Node'a yerleştirir. kubelet limit'i Linux cgroup'lar üzerinden uygular. Probe'ları kubelet periyodik olarak çalıştırır; sonuçları endpoints controller'a yansıtır.

Probe Yaşam Döngüsü — Startup, Liveness, Readiness



startupProbe başarılı olana kadar liveness/readiness çalışmaz. Yavaş başlayan uygulamalar için kritik.

3) Komutlar ve Önemli Flag'ler

Resource ve probe konularıyla en sık kullanılan komutlar:

Komut	Açıklama
kubectl top pods [-n <ns>]	Anlık CPU/RAM kullanımı (metrics-server gerekir).
kubectl top nodes	Node bazında kullanım.
kubectl describe pod <ad>	Probe sonuçları, OOMKilled olayı, restart sayısı.
kubectl get events -A --sort-by=.lastTimestamp tail	Son K8s olayları (Pod restart, OOM, Pull error...).

kubectl set resources deploy/web --requests=cpu=100m --limits=memory=256Mi	Resources hızlıca ayarla.
kubectl autoscale deploy/web --min=2 --max=10 --cpu-percent=60	HPA yaratır (metrics-server gerekir).
kubectl get hpa	HPA listesi ve hedef metrikler.

Probe alanlarında en sık kullanılan parametreler:

Flag	Anlam	Örnek
initialDelaySeconds	İlk probe denemesinden önce bekleme.	5
periodSeconds	Probe'lar arası süre.	10
timeoutSeconds	Tek probe için zaman aşımı.	1
successThreshold	Ardışık kaç başarı 'healthy' sayılır (1).	1
failureThreshold	Ardışık kaç başarısızlık restart/unready.	3
httpGet.path/port	HTTP probe için yol ve port.	/healthz, 8080
tcpSocket.port	TCP probe.	5432
exec.command	Komut çalıştırarak probe.	["cat", "/tmp/healthy"]

4) Declarative — YAML Manifest

Resources + 3 probe entegrasyonlu bir konteyner:

```
containers:
- name: web
  image: web:1.0
  ports: [{ name: http, containerPort: 8080 }]
  resources:
    requests: { cpu: 100m, memory: 128Mi }
    limits: { cpu: 500m, memory: 256Mi }
  startupProbe:
    httpGet: { path: /healthz/startup, port: http }
    failureThreshold: 30 # ≈60 sn (30 * 2 sn)
    periodSeconds: 2
  livenessProbe:
    httpGet: { path: /healthz/live, port: http }
    periodSeconds: 10
    failureThreshold: 3
  readinessProbe:
    httpGet: { path: /healthz/ready, port: http }
    periodSeconds: 5
    failureThreshold: 2
  lifecycle:
    preStop:
      exec: { command: ["sh", "-c", "sleep 5"] }
```

5) İleri Detaylar, Best Practice ve Sık Hatalar

- QoS sınıfları: Guaranteed (requests==limits), Burstable (limits>requests), BestEffort (hiçbiri). Node baskıda BestEffort önce öldürülür.
- CPU birimi: 1 = 1 çekirdek; 500m = 0.5 çekirdek. Bellek: Mi/Gi gibi binary.

- CPU limit throttle yapar, restart yapmaz. Bellek limit ise OOMKilled → restart.
- Liveness probe agresif olursa sürekli restart döngüsü oluşur. Önce readiness ile sınırlandırın; liveness yalnızca "deadlock" gibi gerçek hata durumları için.
- readinessProbe yoksa Service Pod'u hemen endpoint listesine ekler — uygulama henüz hazır değilken trafik alır.
- startupProbe yavaş başlayan uygulamalar (Java, .NET) için zorunludur; liveness/readiness'in agresif failureThreshold'una takılmasını engeller.
- HPA için metrics-server şart: kubectl top çalışmıyorsa HPA da çalışmaz.
- HPA + sabit replicas çakışır: HPA bağlı Deployment'tan spec.replicas alanını kaldırın.
- VPA (Vertical Pod Autoscaler): requests'i öğrenip otomatik ayarlar; üçüncü taraf, bilinmesi iyi.
- Sık hata: resources tanımı eksik Pod scheduler için en alt önceliklidir, Node baskıda ilk öldürülen olur. Her zaman requests koyun.
- Sık hata: HTTP probe Pod hazır değilken bile 200 dönen / path'i kullanmak. Ayrı bir /healthz/ready uç noktası şart.

M9. Ingress (60 dk)

1) Amaç ve Mantık

Ingress, cluster dışından HTTP/HTTPS trafiğini içerideki Service'lere yönlendiren L7 (uygulama katmanı) kuralıdır. Path-based ve host-based routing, TLS terminasyonu, rewrite gibi özellikleri tek bir public IP üzerinden sağlar.

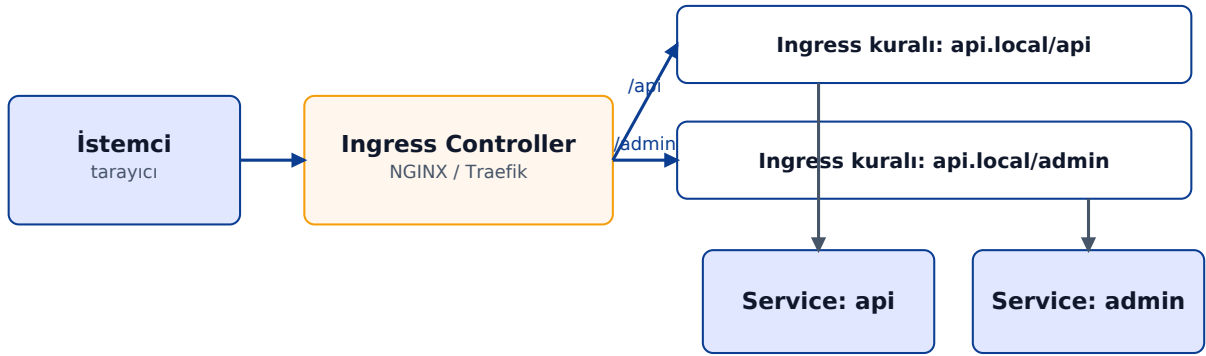
Ingress kuralları Ingress Controller tarafından gerçek bir proxy (NGINX, Traefik, HAProxy, AWS ALB) üzerinde uygulanır. Cluster'da bir Ingress Controller yoksa Ingress nesnesi yaratmak hiçbir şey yapmaz.

Minikube'de minikube addons enable ingress komutu NGINX Ingress Controller'ı devreye alır.

2) Mimarideki Yeri

İstek akışı: Public DNS → Cloud LB → Ingress Controller Pod'u → Ingress kuralları ile eşleşen Service → Endpoints → Pod. Ingress Controller'ın kendisi de bir Pod'dur.

Ingress Topolojisi — Dış İstemciden Pod'a



Ingress = L7 yönlendirme kuralları. Ingress Controller bunları gerçek bir proxy'ye uygular.

3) Komutlar ve Önemli Flag'ler

Ingress için temel komutlar:

Komut	Açıklama
kubectl get ingress -A	Tüm Ingress kurallarını gör.
kubectl describe ing <ad>	Backend Service'leri, ilişkili IP'yi, annotation'ları gösterir.
kubectl get ingressclass	Cluster'da kayıtlı Ingress Controller'lar.
kubectl create ingress web --rule="p layground.local/*=playground:80"	Tek satırla Ingress (1.19+; çoğunlukla YAML tercih edilir).

kubectl annotate ingress web nginx.ingress.kubernetes.io/rewrite-target=/	Annotation ekle (controller-spesifik davranış).
---	---

Ingress spec'inde sık kullanılan alanlar:

Flag	Anlam	Örnek
ingressClassName	Hangi controller işleyecek.	nginx
rules.host	DNS adı; Host header eşleşmesi.	api.example.com
paths.path / pathType	Path ve eşleşme tipi: Exact Prefix ImplementationSpecific.	/api, Prefix
backend.service	Trafik hangi Service'e gidecek.	name: api, port: number 80
tls.hosts / secretName	TLS sertifikası için.	[api.example.com], api-tls
annotations	Controller'a özel ayarlar; standart değildir.	nginx.ingress.kubernetes.io /...

4) Declarative — YAML Manifest

Path-based + TLS örneği:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  tls:
  - hosts: ["web.example.com"]
    secretName: web-tls
  rules:
  - host: web.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service: { name: api, port: { number: 80 } }
      - path: /
        pathType: Prefix
        backend:
          service: { name: web, port: { number: 80 } }
```

5) İleri Detaylar, Best Practice ve Sık Hatalar

- pathType: Exact birebir eşleşme; Prefix dizin sınırına göre prefix; ImplementationSpecific controller'a bırakır.
- TLS: kubernetes.io/tls türünde Secret gerekir (cert + key). Cert-manager + Let's Encrypt üretimde standarttır.
- Annotation'lar controller-spesifik: NGINX'in nginx.ingress.kubernetes.io/... annotation'ları Traefik'te çalışmaz.

- Ingress vs Gateway API: Gateway API (1.27+ GA) Ingress'in yerini almaya hazırlanan yeni modeldir; daha esnek ve role-based.
- defaultBackend: eşleşmeyen istekler için fallback Service.
- External-DNS: Ingress'lerin host'larını otomatik olarak Route53/Cloud DNS'e yazan operator.
- Sık hata: Ingress Controller yok ama Ingress nesnesi yaratıldı — "hiçbir şey çalışmıyor".
- Sık hata: Local'de /etc/hosts'a host eklemeyi unutmak.
- İpucu — debug: `kubectl -n ingress-nginx logs deploy/ingress-nginx-controller`; 404 ve backend hataları orada görünür.

M10. Kapanış Projesi ve Sorun Giderme (90 dk)

Kursta öğrenilen tüm yapılar bu projede bir araya getirilir. Eğitmen, manifest'lere kasıtlı hatalar enjekte eder; kursiyerler logs, describe, events ile sorunu bulur.

Senaryo

- Bir web uygulaması (örn. nginx veya bir .NET MVC uygulaması) Deployment ile çalışır, replikası 3'tür.
- Yapılandırma ConfigMap'ten, parola Secret'tan gelir.
- Veri kalıcılığı PVC ile sağlanır.
- Service (ClusterIP) Pod'ları açığa çıkarır; Ingress public erişim sağlar.
- HPA, CPU kullanımı %60'ı geçince replikayı en fazla 6'ya kadar arttırır.
- Probe'lar (startup + liveness + readiness) doğru yapılandırılmıştır.

Eğitmen Enjekte Edebileceği Hatalar (Troubleshooting)

Komut	Açıklama
Yanlış image tag	Pod ImagePullBackOff'a düşer. Tanı: kubectl describe pod Events bölümü.
Selector mismatch	Service endpoint listesi boş kalır. Tanı: kubectl get endpoints <svc>.
readinessProbe yanlış path	Pod sürekli not ready. Tanı: kubectl describe pod, probe sonuçları.
Bellek limit'i çok düşük	OOMKilled. Tanı: kubectl describe pod Last State: Terminated, Reason: OOMKilled.
ConfigMap'in adı yanlış	Pod CreateContainerConfigError. Tanı: describe pod.
Secret eksik	Pod CreateContainerConfigError. kubectl get secret.
PVC accessMode uyumsuz	PVC Pending. Tanı: kubectl describe pvc.
Yanlış ingressClassName	Ingress controller işlemez; 404. Tanı: kubectl describe ing.

Lab Akışı

- 0-15 dk: Manifest'leri grup halinde gözden geçir; her nesnenin görevini söyle.
- 15-60 dk: Eğitmen 3 hata enjekte eder; gruplar hatayı bulup düzeltir.
- 60-80 dk: Rolling update + rollback + HPA tetikleme demosu.
- 80-90 dk: Sınıfta kısa Q&A; cluster temizliği (kubectl delete ns playground).

Ek A — Komut Hızlı Referans (Cheatsheet)

Komut	Açıklama
kubectl explain <kind>.spec --recursive	Tüm alanların açıklamasını öğren.
kubectl get <kind> -o yaml	Mevcut nesneyi YAML olarak gör (sürüm/anotasyon dahil).
kubectl apply -f file.yaml --dry-run=server	Sunucuda doğrula, uygulama.
kubectl diff -f file.yaml	Mevcut state ile manifest farkı.
kubectl auth can-i <verb> <kind>	RBAC izin kontrolü.
kubectl get all -n <ns> -o name	Sadece isimleri al; script için.
kubectl logs -l app=web --all-containers --since=1h	Bir Service arkasındaki tüm Pod'ların loglarını birlikte oku.
kubectl logs <pod> --previous -c <container>	Crash olan bir önceki konteyner instance'ının son loglarını oku — CrashLoopBackOff teşhisinde ilk komut.
kubectl wait --for=condition=Ready pod -l app=web --timeout=2m	Hazır olana dek bekle (CI/CD için).
kubectl run debug --rm -it --image=nicolaka/netshoot -- sh	Tüm network tool'larıyla geçici debug Pod'u.
kubectl get events --sort-by=.lastTimestamp tail -20	Son olayları kronolojik gör.

Ek B — Sorun Giderme Karar Ağacı

Pod hâlâ Pending ise:

- kubectl describe pod → Events: FailedScheduling? Sebep: yetersiz CPU/RAM, taints, PVC bekliyor.
- kubectl get nodes -o wide → kapasite ve durum.
- PVC pending mi? kubectl describe pvc → StorageClass yok ya da hatalı.

Pod CrashLoopBackOff ise:

- kubectl logs <pod> --previous → bir önceki konteynerin son log'u.
- kubectl describe pod → Last State: Terminated; ExitCode kritik (137=OOM, 1=app hatası).
- command/args yanlış mı? imagePullPolicy?

Service çalışmıyor ise:

- kubectl get endpoints <svc> → boşsa selector/label uyumsuz.
- Pod ready mi? kubectl get pods -l ... READY sütunu.
- kubectl run probe --rm -it --image=busybox -- wget -qO- svc-name → DNS çözülüyor mu?

Image çekilemiyor:

- `kubectl describe pod` → `ErrImagePull` / `ImagePullBackOff`.
- Tag doğru mu? Registry public mi? `imagePullSecrets` gerekli mi?
- Minikube'de minikube image load ile lokal image'ı yükleyin.

Ek C — Kurs Sonrası Yol Haritası

- Helm ile paket yönetimi; Kustomize ile manifest katmanlama.
- HPA + VPA + Cluster Autoscaler üçlüsü.
- NetworkPolicy ve service mesh (Istio/Linkerd).
- StatefulSet, Job, CronJob, DaemonSet.
- RBAC, ServiceAccount, OIDC ile yetkilendirme.
- Operator pattern ve Custom Resource Definition (CRD).
- GitOps: ArgoCD / Flux ile manifest senkronizasyonu.
- Sertifikasyon: CKAD (geliştiriciler), CKA (operatörler), CKS (güvenlik).